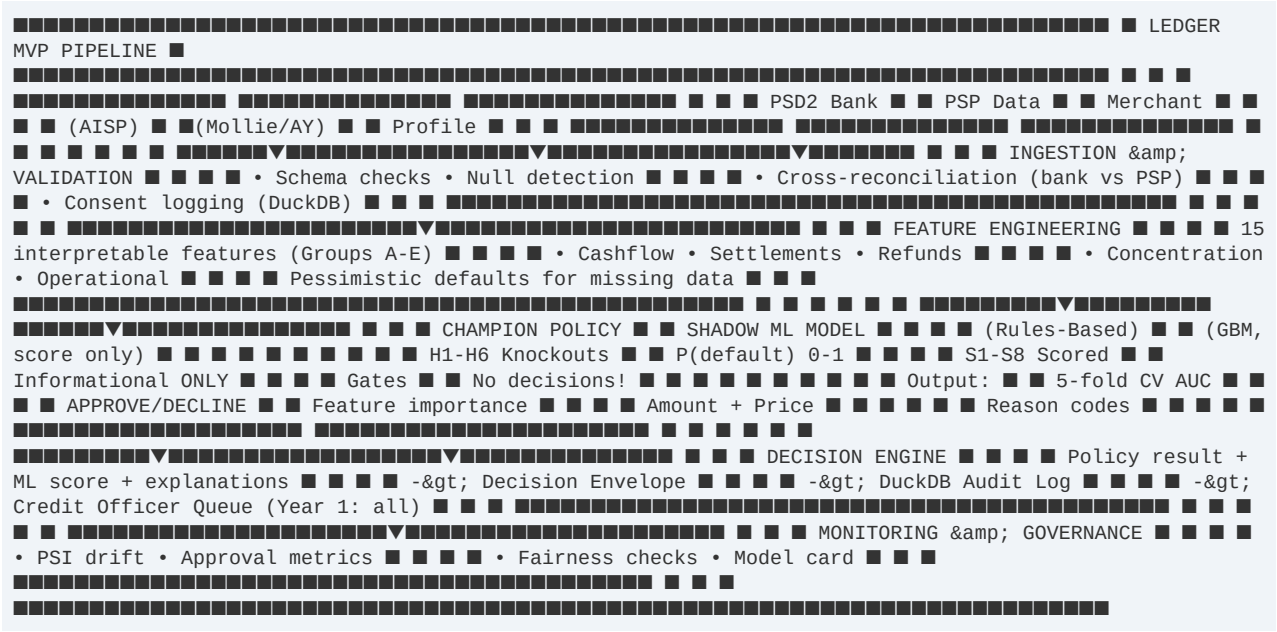
LEDGER MVP

Pipeline Architecture & Code Summary

This document explains how the Ledger MVP underwriting system works end-to-end: from data ingestion through feature engineering, credit policy, shadow ML, decision output, and monitoring.

1. High-Level Pipeline Architecture



2. Data Flow: Ingestion to Decision

Step 1: Synthetic Data Generation

Module: `data/synthetic_gen.py`
Generates 200 synthetic Dutch e-commerce merchants with 12 months of transaction history. Each merchant gets bank transactions (PSP settlements, supplier payments, ad spend) and order-level PSP data (with refunds/chargebacks). 5% of merchants are flagged as 'bad' — these have higher volatility, more refunds, and more chargebacks. All data is seeded (seed=42) for reproducibility and saved as parquet files.

Step 2: Data Ingestion & Validation

Modules: `ingestion/bank_source.py`, `ingestion/psp_source.py`
Both inherit from an abstract DataSource class that enforces a load → validate → log pattern. Validation checks: required columns present, null rates below 1% for critical fields, minimum 180 days of history. In production, these would connect to AISP and PSP APIs; in the MVP, they read parquet files.

Step 3: Cross-Source Reconciliation

Module: `ingestion/reconciliation.py`
For each merchant, compares total bank credits labelled 'psp_settlement' against total PSP net settled amounts. If the absolute delta exceeds 20%, the merchant is flagged with RECON_FAIL — a fraud/data-integrity signal. This is a hard knock-out in the credit policy.

Step 4: Feature Engineering

Module: `features/pipeline.py`
Computes 15 interpretable features per merchant from raw bank + PSP data. Features are organized into 5 groups: (A) cashflow stability, (B) settlement delays, (C) refunds & chargebacks, (D) concentration & seasonality, (E) operational/ad spend. Every feature has an explicit pessimistic default for missing data — the system assumes worst-case rather than dropping the merchant.

Source	Features Derived	Count
Bank transactions	monthly_net_cashflow, revenue_volatility, negative_balance_pct, ad_spend_ratio, supplier_punctuality	5
PSP transactions	settlement_delay (p95, median), refund_rate, chargeback_rate, refund_trend, platform_concentration	6
Bank × PSP cross	bank_psp_recon_delta	1
Bank (monthly agg)	seasonality_index, monthly_gmv_avg_6m	2
Derived (application)	net_cashflow_coverage	1

Step 5: Champion Credit Policy

Module: `policy/credit_policy.py`
A deterministic, fully interpretable rules-based model. Processes in strict order:

- **Hard knock-outs (H1–H6):** Any single failure → immediate DECLINE. Tests trading history, KvK status, reconciliation, balance health, sanctions, minimum GMV.
- **Scored gates (S1–S8):** Each of 8 features is scored green (0 pts), amber (1 pt), or red (2 pts). Includes inverted gates where lower = worse (cashflow coverage, supplier punctuality).
- **Decision logic:** ≥3 reds → DECLINE; ≥2 reds or ≥5 total → MANUAL_REVIEW; else → APPROVE. Year 1: ALL approvals also flagged for manual credit officer review.

• **Amount cap:** min(requested, cashflow × 0.33 × tenor, GMV × 2.5, EUR 25k pilot cap). Rounded to nearest EUR 100.

• **Pricing band:** A (11%) for clean profiles, B (12.5%) default, C (14%) for higher risk. Matches business plan's 11–14% APR range.

• **Output:** Decision envelope with reason codes (machine-readable) and explanations (human-readable sentences).

Step 6: Shadow ML Scoring

Module: `models/train_shadow.py`
A GradientBoostingClassifier trained on the same 15 features used by the champion policy. Target: synthetic 'is_bad' label (proxy for 30+ DPD default). Evaluated via 5-fold cross-validation (AUC + Brier score). Produces a P(default) score between 0 and 1. **Critical: this model produces scores ONLY and never makes or influences lending decisions.** It is logged alongside the champion decision for future comparison and calibration. No ML superiority claims are made without real repayment data.

Step 7: Decision Assembly & Audit

Module: `decisioning/decision_engine.py`
Combines: (1) champion policy result (the actual decision), (2) shadow ML score (informational note), (3) human-readable explanations, and (4) manual-review flags. The complete decision envelope is persisted to DuckDB with the full feature vector as JSON — providing a complete audit trail for every credit decision.

3. Model Overview: Champion vs Shadow

Aspect	Champion (Rules-Based)	Shadow (ML)
Type	Deterministic rules + scored gates	GradientBoostingClassifier
Authority	Makes the lending recommendation	Informational score only — ZERO authority
Interpretability	Fully transparent: every rule + threshold visible	Feature importances; SHAP planned for production
Output	APPROVE / DECLINE / MANUAL_REVIEW + amount + reason codes	p(default) ∈ [0, 1]
Human review	Year 1: ALL loans reviewed by credit officer	Not reviewed; logged for comparison
Training data	N/A (expert rules from business plan)	Synthetic labels (is_bad_synthetic)
When updated	Thresholds tuned based on portfolio experience	Retrained after 200+ real repayment outcomes
Governance	Policy signed off by Head of Credit	Model card + shadow-only restriction
Risk	May miss non-linear patterns	Trained on synthetic data — performance unknown on real merchants

> *The champion-vs-shadow architecture follows the business plan's staged approach: 'Phase 2 trains a model such as gradient boosting in shadow mode, where it produces scores but does not yet make lending decisions. Phase 3 may use machine learning as the primary score, but only with human override, explanations, performance monitoring and adverse-impact testing.'*

4. Key Design Decisions & Rationale

Decision	Rationale
Pessimistic defaults for missing features	Conservative underwriting: assume worst plausible value rather than dropping applications. Protects portfolio qu
Same feature set for champion & shadow ML	Ensures apples-to-apples comparison. If ML later outperforms rules, it's because of the model, not different data
DuckDB for audit logging	Lightweight, local, SQL-compatible. No external database dependency for MVP. Easily queryable for monitoring
Year-1 pilot cap at EUR 25,000	Business plan specifies 'Keep Year-1 tickets at EUR 10–25k'. Limits exposure while collecting repayment data.
All loans to manual review in Year 1	Business plan: 'reviewed by a credit officer for every loan'. The system recommends; the human decides.
Cross-reconciliation as hard knock-out	Bank-PSP mismatch > 20% is a fraud/integrity signal too severe to override via scoring. Forces data quality.
Parquet for data storage	Columnar format with compression. Fast reads for analytics. Easy migration to cloud later.
No SHAP in MVP	Feature importances are global, not per-prediction. SHAP adds complexity; planned for production.
Synthetic data with known 'bad' merchants	Allows testing all policy branches (approve, decline, manual review) without real data. Seeded for reproducibility
3-way amount cap (cashflow × ratio, GMV anchor, product cap)	Prevents over-lending. Cashflow ensures debt-service ability, GMV anchors to business size, product cap limits

5. End-to-End Code Walkthrough

The `run_pipeline.py` orchestrator executes the following sequence:

Step	Code	What Happens
1. Load data	BankTransactionSource().load_and_validate() PSPTransactionSource().load_and_validate() pd.read_parquet('merchants/applications')	Reads parquet files, validates schemas, logs results. 200 merchants, ~500k+ bank txns,
2. Compute features	compute_features(mid, bank, psp, merchant, app) for each merchant	Filters raw data, computes 15 features with pessimistic defaults. Returns pd.Series per m
3. Train shadow	prepare_training_data(features, merchants) train_shadow_model(X, y, 'gbm')	Merges feature matrix with synthetic labels. Trains GBM (200 trees, depth 4). 5-fold CV.
4. Run policy	credit_policy(feat_dict) for each merchant	Knockouts → scored gates → red/amber count → decision → amount → pricing → reasc
5. Score ML	score_merchant(feat_dict, model) for each merchant	Produces P(default) from shadow GBM. Informational only.
6. Assemble	make_decision(mid, features, policy, ml_score)	Builds decision envelope. Adds ML narrative. Logs to DuckDB with full feature vector.
7. Report	decisions_df summary + example envelope	Prints approval/decline counts, avg amounts, pricing distribution, example decision.

6. Monitoring in Production

Once deployed, three monitoring layers protect portfolio quality:

Layer	Module	What It Checks	Alert Threshold
Feature drift	monitoring/drift.py	PSI per feature vs reference population	PSI > 0.2 → significant drift
Portfolio metrics	monitoring/metrics.py	Approval rate, decline reasons, ML score distribution	Approval rate outside 40–60%
Fairness	monitoring/fairness.py	Approval rate by sector and platform count	Sector rate < 80% of overall

All monitoring reads from the DuckDB decision log — the same audit trail that stores every decision. This means monitoring is always consistent with actual decisions and can be run retroactively.